PACT

Volume 1 - Product requirements, workflows and acceptance criteria

Document type: Implementation-ready PRD and functional requirements. Audience: founder, product designer, smart contract/API engineers, security reviewer and Codex. Status: specification; no deployed implementation is claimed. Companion: Volume 2 supplies detailed engineering contracts, data models, operations and delivery plan.

1. Executive brief

Pact is a programmable authority and economic accountability layer for autonomous agents. Principals grant narrowly scoped authority. Agents submit economic intents. Pact enforces rules at controlled wallet and API boundaries, measures the total liability of an action, reserves authority, records mutually agreed obligations, verifies outcomes under named rules, settles funds and produces a linked economic receipt.

Core differentiator: commitment-aware authority. A 1,000 USDC/month agreement for six months creates up to 6,000 USDC of contractual payment liability, even if the first charge is only 1,000. An agent with 500 of delegated capacity must not be allowed to sign it.

Success definition

• A counterparty can verify a specific agent proposal against a live onchain reservation without trusting the Pact website.

• A Pact-controlled wallet or vaulted API key refuses disallowed execution even when the agent or model ignores instructions.

• Every completed obligation links its grant, policy version, approval, terms, evidence, verifier result and settlement.

• A second developer can integrate one agent using the SDK and reproduce the end-to-end demo.

2. Users and jobs

User	Need	Primary surface
Principal / organization admin	Create agents, define scope, approve exceptions, track live commitments and revoke new authority.	Console
Autonomous agent developer	Integrate tools and receive explicit quote/decision/execution states.	SDK and MCP
Agent operator	Operate agents without holding principal signing key or raw upstream provider key.	SDK/Gateway
Counterparty	Know whether agent may make this exact offer and what was agreed.	Proof verifier/Explorer
Verifier / resolver	Assess evidence and submit attributable decision under terms.	Verifier API/Console
Auditor	Reconstruct who authorized what and what economic outcome occurred.	Explorer/export

3. Product scope and release tiers

Tier	Included	Excluded or deferred
MVP	Single supported stablecoin; root/child grant; policy engine; ERC-20 transfer; fixed-price and milestone agreement; one vaulted API integration; named verifier; approval; escrow; receipts; SDK, MCP, Console, Explorer.	Unreviewed arbitrary contract calls, cross-chain authority, credit underwriting, automated legal enforcement, marketplace, universal oracle.
Post-MVP	Recurring agreements, more adapters/assets/providers, richer disputes, privacy-preserving proofs and standards bridges.	Only after each adapter can bound exposure.
Not promised	Control of agents using outside wallets/raw keys; truth of subjective evidence without trusted method; legal validity everywhere.	Explicitly disclose in interface and docs.

4. Functional requirements: identity and authority

FR-001 A principal creates a grant binding agent identity, operational key, policy hash, validity and ceiling. Acceptance: Invalid signature/nonce/expiry cannot create a grant.

FR-002 A grant may delegate a narrower subset with bounded depth, target, time and allocated capacity. Acceptance: Child request exceeding any parent field fails.

FR-003 Principal revocation invalidates future descendant actions and increments the relevant epoch. Acceptance: New reservation fails after revocation; existing obligation retains settlement rights.

FR-004 Agent key rotation changes key epoch; old signatures cease authorizing new intents. Acceptance: Previous key cannot propose a new action.

FR-005 Principal sees currently available, allocated, reserved, committed and settled amounts separately. Acceptance: Displayed totals reconcile with contract reads and event log.

5. Functional requirements: policy and execution

FR-010 Typed intent must identify agent/grant, action kind, chain, target, asset, amount, payload hash, nonce and deadline. Acceptance: Free-form text alone cannot authorize execution.

FR-011 Pact evaluates access, financial, time, delegation and economic rules with DENY / REQUIRE_APPROVAL / ALLOW. Acceptance: Equivalent inputs produce equivalent decisions with reason codes.

FR-012 Known adapters compute maximum bounded exposure and attach assumptions, version and validity. Acceptance: Unknown or unbounded actions fail closed.

FR-013 Principal approval signs exact intent hash, quote hash, terms hash, amount, liability, chain, expiry and nonce. Acceptance: Changed terms invalidate old approval.

FR-014 PactAccount rechecks grant and action scope in the same transaction that executes an approved action. Acceptance: Engine approval alone cannot move controlled funds.

FR-015 Gateway keeps upstream credentials, checks endpoint/method/payload, reserves budget and logs redacted result. Acceptance: Agent never receives provider secret or bypasses policy using gateway routes.

FR-016 Provider call and chain operation use idempotency and terminal reconciliation states. Acceptance: Retry does not duplicate execution or count spend twice.

6. Functional requirements: proof, agreement and settlement

FR-020 Action-specific Proof of Authority binds grant, policy, terms, maximum liability and live capacity reservation. Acceptance: Counterparty verifies proof without Pact API; stale reservation fails.

FR-021 Proof of Obligation binds both parties to the same versioned terms and original authority. Acceptance: Terms cannot be edited after acceptance without fresh signatures.

FR-022 Terms specify parties, deliverable, milestones, asset, gross liability, funding, verifier, deadline, challenge and exit rights. Acceptance: Missing mandatory terms prevent agreement activation.

FR-023 SettlementVault separates committed authority from deposited cash and releases only under permitted state transitions. Acceptance: Underfunded or duplicate payout reverts.

FR-024 Evidence is hashed, stored under access control, and evaluated by named deterministic or attesting verifier. Acceptance: Explorer labels the verification trust model.

FR-025 Challenge holds contested funds for specified period and records resolver outcome. Acceptance: Dispute cannot silently settle before resolution or agreed timeout.

FR-026 Economic receipt links full lifecycle and distinguishes contract proof from third-party attestations. Acceptance: Tampered evidence hash or missing settlement detected by independent verifier.

7. Core user workflows

7.1 Set up principal and agent

Authenticate principal -> bind wallet/approved signature method -> create controlled account -> register or bind agent identity -> choose operational key -> set root grant and policy -> show readable maximum cash and future liability -> principal signs -> contract confirms -> agent receives scoped SDK credentials. A login session by itself must not create financial authority.

7.2 Delegate authority

Agent A proposes child scope -> validator checks each field is narrower -> ledger allocates child capacity and prevents parent reusing it -> principal or authorized parent signs -> descendant becomes active. Revocation blocks new actions along the tree. Capacity is released from child allocation only according to defined revocation and active-commitment handling.

7.3 Propose and execute

Agent submits typed intent -> adapter quotes cash movement and maximum liability -> engine evaluates policy -> DENY includes reasons, APPROVAL includes exact human signing request, ALLOW proceeds to onchain reservation -> account revalidates and executes. Neither ALLOW nor a signed approval is displayed as a confirmed payment.

7.4 Agree, deliver and settle

Agent proposes terms -> reserve capacity -> counterparty independently verifies authority -> both sign terms -> vault receives deposit -> obligation activates -> provider submits milestone evidence -> verifier returns attributable result -> challenge window passes or dispute resolves -> vault pays/refunds -> receipt finalizes.

7.5 Change and recovery

Renegotiating price, term, counterparty or verifier requires a new terms hash and fresh approvals. If provider/API/chain action fails, reconcile before releasing capacity. If an agent key is compromised, rotate or revoke; show all outstanding obligations and their settlement rights.

8. Screen and experience requirements

Screen	Required content / actions
Landing and developer entry	Plain-English thesis, working demo link, SDK/API docs and onchain verifiability; no unfounded claims.
Console overview	Capacity ledger, live commitments, pending approval, expiring grant, failed action and active dispute.
Agents and grants	Agent identity, key state, parent tree, capabilities, allocated ceiling, expiration, rotate/revoke.
Policy editor	Typed conditions, readable summary, version diff and realistic dry-run; no silent wildcard expansion.
Approval detail	Counterparty, exact terms, cash now, gross liability, duration, escrow amount and signature expiry.
Obligation detail	Terms, funding, milestones, verifier, evidence, challenge status, final settlement.
Explorer proof	Grant/reservation status, liability, disclosed terms, evidence trust label and independent verify action.

Design direction: minimal radius, generous space, fine rules, clear typography, no decorative gradients. Use neutral visual hierarchy to distinguish capacity from funds actually locked and from money paid.

9. Security, privacy and accessibility requirements

• No raw private keys or API credentials in browser bundles, agent environments, logs, receipts or MCP outputs.

• All privileged mutations require scoped service authentication and the required principal/party signature or onchain caller authority.

• Encrypt private policy/evidence, apply tenant isolation, set retention/deletion rules for offchain data while preserving necessary chain anchors.

• Apply rate limits, request size caps, SSRF and redirect controls to gateway; treat retrieved evidence as untrusted input.

• Console must support keyboard navigation, visible focus, readable contrast, clear error messages and non-color status cues.

• Emergency pause must be narrow: stop new reservations/execution while permitting safe exits and legally defined escrow resolutions where feasible.

10. Nonfunctional requirements and observability

Area	Initial requirement	Verification
Consistency	Contracts authoritative; indexer rebuildable; API exposes pending/confirmed/final states.	Kill indexer, rebuild and compare state.
Latency	Quote target under 2 seconds for supported local actions in healthy environment; onchain time reported separately.	Benchmark at controlled load, report p50/p95.
Reliability	Every mutation idempotent or safely reconciled; operations have terminal or actionable recovery state.	Inject RPC/provider timeouts and retries.
Security	Independent contract review and threat-model review before production funds.	Document findings and fixes.
Audit	Trace ID links decision, signatures, transaction, evidence and settlement without logging secrets.	Trace one demo from start to finish.
Portability	TS and Python SDKs agree on hashes, amounts, schema versions and error codes.	Shared fixture suite.

Performance targets are engineering targets, not promises about Monad finality or external providers. Onchain confirmation, evidence checks and challenge windows have distinct timing.

11. Metropolis demonstration and submission

The demo should show: an unauthorized API method denied; an over-limit payment denied; a 1,000/month for 24 months offer denied for 24,000 liability; revised valid terms requiring human approval; counterparty verifying reserved authority; signed obligation and escrow; verifier outcome, settlement and independently checked receipt. Demonstrate a child with only 500 capacity failing a 6,000 obligation before granting it adequate scope.

Show contract addresses, explorer transactions, code provenance during the build window, tests, a reproducible setup, a concise video and a live app. Metropolis lists a build window and working-product submission on its official page; check current dates and prize/bounty rules before submission.

12. Open decisions and product gates

Decision	Default for demo	Decision owner / gate
Network and asset	Monad testnet and one verified token with explicit decimals.	Engineering verifies current addresses.
Principal signing	Wallet EIP-712; passkey path only after target account verification.	Security/product.
Verifier	Named signed attestation with clear challenge window.	Product chooses trust model.
Disputes	Named resolver and fixed timeout; no hidden unilateral override.	Product/security.
Privacy	Public hashes and scoped disclosures.	Product/legal review.
API provider	One provider with bounded cost and scoped credential.	Engineering selects provider.
Sponsor bounties	Optional after exact rules checked.	Founder prioritizes credible fit.

13. Product acceptance checklist

- All FR-001 through FR-026 behaviors demonstrated or explicitly marked deferred.
- A third party independently verifies action-bound authority and final receipt.
- Agent does not hold an unrestricted account key or raw provider secret.
- Code, contracts, SDK examples and UI reflect the same accounting model.
- A failed or challenged operation never appears as settled.
- Threat boundaries and verifier assumptions are disclosed to users and judges.

Primary sources: https://monad.xyz/metropolis ; https://docs.monad.xyz/ ; https://eips.ethereum.org/EIPS/eip-8004 ; https://eips.ethereum.org/EIPS/eip-7715 ; https://ap2-protocol.org/ap2/agent_authorization/ ; https://modelcontextprotocol.io/specification/2026-07-28 . Confirm their current state during build.